

Report: Q26

Travelling Salesperson Vs Bellman-Ford (DP Comparison)

Title

Comparative Study of Travelling Salesperson Problem Using Dynamic Programming and Bellman-Ford Shortest Path Algorithm

Aim

To compare the Dynamic Programming solution for the Travelling Salesperson Problem (TSP) with the Bellman-Ford shortest path algorithm and analyze their computational complexity, scalability, and practical applications.

Problem Statement

The objective is to analyze two important graph optimization problems:

1. Travelling Salesperson Problem (TSP)
2. Bellman-Ford Shortest Path Problem

And compare their solution techniques and performance characteristics.

Introduction

Graph optimization is widely used in transportation systems, communication networks, logistics, and route planning. Different optimization objectives lead to different algorithms.

The Travelling Salesperson Problem seeks the minimum-cost tour that visits every city exactly once and returns to the starting city.

Bellman-Ford computes the shortest paths from a source node to all other nodes in a weighted graph.

Although both involve path optimization, their complexity and problem structures differ significantly.

Travelling Salesperson Problem (TSP)

Objective

Find the shortest possible route that:

- Starts from a city
- Visits every city exactly once
- Returns to the starting city

Dynamic Programming Solution

The Held-Karp algorithm uses:

- Bit masking
- Memorization
- State-space optimization

State Representation:

DP (mask, city)

Where:

- mask represents visited cities
- city represents current location

Example Result

For the given graph:

City Pair	Cost
0-1	10
0-2	15
0-3	20
1-2	35
1-3	25
2-3	30

Optimal Tour:

$0 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 0$

Minimum Cost:

80

Bellman-Ford Algorithm

Objective

Determine the shortest path from a source vertex to all other vertices.

Working Principle

Bellman-Ford repeatedly relaxes edges.

Distance updates formula:

$\text{Distance}[v] = \min (\text{Distance}[v], \text{Distance}[u] + \text{Weight})$

The algorithm performs:

$V - 1$ iterations

Where V is the number of vertices.

Example Result

Source Vertex = 0

Shortest Distances:

Vertex	Distance
0	0
1	10
2	15
3	35

Complexity Comparison

Algorithm	Time Complexity	Space Complexity
TSP (DP)	$O(n^2 \times 2^n)$	$O(n \times 2^n)$
Bellman-Ford	$O(VE)$	$O(V)$

Exponential Vs Polynomial Growth

TSP

Number of states:

$$n \times 2^n$$

Examples:

Cities	Approximate States
10	10,240
15	491,520
20	20,971,520
25	838,860,800

The state space grows exponentially.

Bellman-Ford

Example:

$$V = 1000$$

$$E = 5000$$

Operations:

$$V \times E$$

$$= 1000 \times 5000$$

$$= 5,000,000$$

Growth remains polynomial and manageable.

Scalability Analysis

TSP Challenges

- a) Exponential growth in states.
- b) High memory requirements.
- c) Long execution times.
- d) Difficult to apply for large city networks.

Bellman-Ford Advantages

1. Handles large networks efficiently.
2. Supports negative edge weights.
3. Suitable for real-world routing systems.

Routing Applications

TSP Applications

1. Delivery Optimization
2. Vehicle Routing
3. Warehouse Robots
4. Drone Path Planning
5. School Bus Scheduling

Bellman-Ford Applications

1. Computer Network Routing
2. GPS Navigation
3. Traffic Management
4. Telecommunication Networks
5. Transportation Systems

Dynamic Programming Limitations in TSP

1. State explosion problem.
2. Exponential memory requirements.
3. Poor scalability for large datasets.
4. Computationally expensive despite optimization.
5. Often replaced by approximation algorithms for large instances.

Insights from the Comparison

- TSP focuses on finding an optimal tour.
- Bellman-Ford focuses on shortest paths from a source.
- Dynamic Programming improves TSP but cannot eliminate exponential complexity.
- Bellman-Ford remains practical for large graphs due to polynomial complexity.
- Problem nature greatly influences algorithm selection.

Conclusion

The Travelling Salesperson Problem and Bellman-Ford algorithm solve fundamentally different optimization problems. Dynamic Programming enables TSP to find optimal tours but suffers from exponential growth in computational requirements. Bellman-Ford efficiently computes shortest paths in polynomial time and scales well to large networks. This comparison highlights that although Dynamic Programming provides optimal solutions, its effectiveness is limited when the state space grows exponentially. Bellman-Ford remains a more practical choice for large-scale routing applications.